ASSIGNMENT 1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1 | Bloom's Level K5 – Evaluate | Total Marks: 50

Student name	B.Praneeth
Register number	192472222
Course / Section	DSA0103-C++
GitHub repository	https://github.com/praneethbacchu/DSA0103-C-.git
Compiler	g++ 13.3.0 (Ubuntu 24.04)
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file
Submission date	_____

How to read this report

Every program in this report was compiled with g++ 13.3.0 using -std=c++17 -Wall -Wextra and executed; all outputs shown are the real outputs. The report follows Tasks 1–12 in order. Source code for every program is in Appendix A and is stored one folder per task in the GitHub repository.

Assumptions used throughout: money is stored in integer paise (Rs 1 = 100 paise); hour is an integer 0–23 (a journey at 10:00 exactly is off-peak, 20:00 exactly is off-peak, because peak is the half-open range [8,10) and [17,20)); the card must keep at least Rs 50 (5000 paise) after every journey; fare is decided by the farthest zone travelled.

Task 1 – Understand the Problem and Identify the Objects

1.1 Objects, data and behaviour

Object	Data (attributes)	Behaviour (methods)	Why it is an object
SmartCard	balance (paise), minimum balance rule	canTravel(fare), deduct(fare), recharge(amount), balance()	It owns the money; the "never below Rs 50" rule must live next to the balance so no one can bypass it.
Passenger	category (normal / student / senior)	concessionPercent()	Concession depends on who travels, not on the journey.
Journey	zone, hour, weekend flag	validate()	Groups the inputs of one fare calculation and rejects bad input (hour 25, zone 3).
Zone	Inner, Middle, Outer (Airport later)	none (enum class)	A closed set of named values; an enum class prevents mixing zones with plain ints.
FareTable	base fares, surcharge %, peak hours	fare(zone, time), isPeak(hour)	All fare numbers in one place so a revision is a one-line change.

1.2 Procedural versus object-oriented style

In a procedural version the balance is a global variable and fare(), canTravel() and deduct() are free functions that all read and write it. The data and the rules about the data are separate, so any function can set balance = -999 (this actually happened in the first gate software). In the object-oriented version the balance is a private member of SmartCard and can only change through deduct() and recharge(), which enforce the rule. The behaviour moves to the data it protects. Task 7 demonstrates both versions with real output.

1.3 OOP principles and where they appear

Principle	Where it appears in this system	Justification
Encapsulation	SmartCard::balance_ is private; changed only by deduct()/recharge()	Makes a negative or sub-Rs-50 balance impossible by construction.
Abstraction	Gate code calls card.deduct(fare) and table.fare(zone, time) without knowing how	The fare algorithm can change (switch to table lookup) without touching the gate.
Inheritance	A base class Concession with derived Student and Senior; or Ticket / MonthlyPass	Shared interface for different passenger or ticket types (used lightly; composition is preferred for Pass in Task 12).
Polymorphism	virtual long long Concession::apply(long long fare) overridden for Student, Senior	The gate calls apply() without if-else on the passenger type.

1.4 Rules every correct journey must satisfy

- R1: Balance after a journey >= 5000 paise (Rs 50); a journey that would break this is denied and nothing is deducted.
- R2: Balance is never negative; recharge amount must be > 0.
- R3: Fare = base fare of the farthest zone (Rs 10/20/30), +20% in peak hours [8,10) and [17,20).
- R4: Peak boundaries are consistent: 8:00 and 17:00 are peak; 10:00 and 20:00 are off-peak.
- R5: Fare and balance are exact integers in paise; no floating-point money.
- R6: Invalid input (zone outside the enum, hour outside 0–23, letters instead of numbers) is rejected, never converted silently.
- R7: Every fare value is defined in exactly one place.

Alternative concept and complexity analysis – procedural style

Solution: global long long balance; global constants; free functions fare(zone, peak), canTravel(f), deduct(f). Time per journey: one switch, one multiply/divide, one compare, one subtract – O(1) in both styles. Memory per card: one long long (8 bytes) in both – O(1); a class adds no per-object overhead for non-virtual members (sizeof(SmartCard) = 8). The difference is maintainability: in the procedural version a changed rule (e.g. minimum balance) must be checked in every function that touches balance, whereas in the OOP version it is changed in one constant inside SmartCard. A measured proof: in Task 7 the procedural program accepts balance = -999, the class version cannot compile that line.

Task 2 – Tokens, Keywords, Identifiers and Constants

2.1 The five kinds of tokens (from Task 6 code)

Token kind	Examples from the program
Keywords	enum, class, constexpr, long, long, bool, return, switch, case, break, if, else, for, const
Identifiers	Zone, BASE_PAISE, fare, canTravel, balance, trips, MIN_BALANCE_PAISE
Constants (literals)	1000, 20, 5000, 100, '.', '\n', "Fare Rs ", true, false
Operators	=, +=, -=, /, %, >=, <<, ::, ++
Punctuators	{ } ( ) [ ] ; , :

2.2 #define, const and constexpr

#define is a text substitution by the preprocessor: no type, no scope, no debugger symbol, and #define SQUARE(x) x*x gives wrong results for SQUARE(a+1). const gives a typed, scoped read-only object, but its value may be computed at run time. constexpr guarantees the value is known at compile time, so it can be used for array sizes and in static_assert, and the compiler can fold it into the machine code. For the fare table, constexpr is preferred because fares are fixed numbers, they must be typed (long long paise), and they can size arrays.

2.3 enum versus enum class

A plain enum leaks its names into the enclosing scope (Inner could clash with another Inner) and converts implicitly to int, so zone == 2 or zone + 1 compiles silently. An enum class keeps names scoped (Zone::Inner) and refuses implicit conversion to int, so passing 7 as a Zone or comparing a Zone with an hour is a compile error. This catches the "unknown zone" bug at compile time.

2.4 Naming rules and conventions

- Identifiers may contain letters, digits and underscore, must not start with a digit, are case-sensitive and must not be keywords.
- Avoid names starting with underscore + capital letter or containing a double underscore (reserved).
- Convention used: UPPER_SNAKE_CASE for constants (MIN_BALANCE_PAISE), PascalCase for types (SmartCard, Zone), camelCase for functions (canTravel), trailing underscore for private members (balance_).

2.5 The single set of constants used

```
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, 20, 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;      // Rs 50
```

Alternative concept and complexity analysis – #define macros

Solution: #define INNER_FARE 1000, #define PEAK 20, #define MIN_BAL 5000. Type checking: macros are untyped text, so INNER_FARE could be used as a double or int by accident and no error is reported; constexpr carries the type long long. Scope: a macro ignores namespaces and class scope and is visible everywhere after the definition, which can silently replace an identifier in unrelated code; constexpr obeys scope. Memory: a macro occupies no storage; a constexpr scalar usually occupies none either because the compiler substitutes the value (the 3-element constexpr array occupies 12 bytes if indexed with a run-time index). Running time: the same – after compilation both become immediate constants, O(1), no run-time cost. Conclusion: no performance difference, but constexpr + enum class is safer.

Task 3 – Data Types and Type Compatibility

3.1 Data-type table (sizes measured with sizeof on this system)

Value	Type chosen	sizeof	Range / reason
Fare, balance, recharge	long long (paise)	8	±9.22 x 10^18; exact integers
Base fare table entries	int (constexpr)	4	max 6000, small
Hour of day	int	4	0–23 (could be unsigned char, int avoids conversions)
Zone	enum class Zone	4	closed set, type-safe
Peak / weekend flags	bool	1	true/false
Journey counter	long long	8	3,000,000 fits int, but total revenue does not
Surcharge %	int	4	20
Calculation shortcut	auto	deduced	only where the type is obvious from the right side

Measured (Task03/types.cpp): bool 1, char 1, int 4, long long 8, unsigned 4, float 4, double 8. INT_MAX = 2147483647, UINT_MAX = 4294967295, LLONG_MAX = 9223372036854775807.

3.2 Why integer paise and not double rupees

Decimal fractions such as 0.10 have no exact binary representation, so a double stores an approximation. Measured result of adding Rs 0.10 n times:

n	double result (Rs)	exact (Rs)	error
10	1	1	-1.11e-16
1,000	99.9999999999986	100	-1.41e-12
100,000	10000.0000000188	10000	1.88e-08
3,000,000	300000.000001957	300000	1.96e-06

Also (0.1 + 0.2 == 0.3) evaluates to false. The error grows with n; in a gate that compares balance - fare >= 50.00 a value like 49.99999999 would wrongly deny a journey. With integer paise the sum is exact for every n.

3.3 Type compatibility rules

In a mixed expression the usual arithmetic conversions apply: types smaller than int are promoted to int; if one operand is double the other becomes double; bool converts to 0 or 1 in arithmetic; a signed operand is converted to unsigned when the unsigned type has the same or higher rank. So int + double → double, bool + int → int (true + 2 = 3), and int < unsigned → both treated as unsigned.

3.4 Ranges and the network total

Worst-case day: 3,000,000 journeys x Rs 30 = 9,000,000,000 paise. INT_MAX is 2,147,483,647, so int is NOT enough (measured: "fits int? false"). long long is required. This is the overflow problem; resolution: declare all accumulators as long long.

Alternative concept and complexity analysis – double rupees

Solution: double balance = 100.00; fare 36.0; compare balance - fare >= 50.0 with a tolerance such as 1e-9. Time per addition is O(1) for both (floating-point add, measured 240 ms vs 31 ms for 10^8 additions at -O1 including int→double conversion, so integer addition is faster in practice but both are O(1) per operation). Space per value is 8 bytes in both. The rounding error grows with n (table above: 1e-16 at n=10 to 2e-6 at n=3,000,000), which is O(n)-growth of error in the worst case, whereas the paise error is exactly zero.

Task 4 – Control Implicit Conversions

4.1 Where implicit conversions occur

- Integer division: 7 / 2 is performed on two ints and gives 3; storing in a double afterwards only converts 3 to 3.0. In fare logic, base * 20 / 100 is safe in integer paise as long as multiplication is done before division.
- Integer-to-double: int mixed with double in an expression is converted to double (e.g. f / 100.0).
- Signed vs unsigned: -1 converted to unsigned becomes 4294967295, so (s < u) is false. The compiler warns with -Wsign-compare. A journey counter declared unsigned compared with a negative int would be wrong.
- Narrowing: int x = 3.7 silently truncates to 3, but int x{3.7}; is a compile error, so brace initialisation prevents it.
- static_cast is required when the conversion is intentional or the compiler will not do it: static_cast<double>(a) / b, or static_cast<int>(zone) to index an array.

4.2 Predict the Output

```
#include <iostream>

int main()
{
```

```
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (write before running):
(write your own prediction here BEFORE running the program)

Actual output (g++ 13.3.0):

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning received: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare].

4.3 Explanation of each line

Output	Rule involved
3	a / b is integer division (7/2 = 3, truncation toward zero); the int 3 is then converted to double 3.0 and printed as 3.
3.5	static_cast<double>(a) makes the numerator double, b is converted to double (usual arithmetic conversion); 7.0/2.0 = 3.5.
C	char ch = 'A' + 2: 'A' (65) is promoted to int, 65 + 2 = 67, narrowed back to char 67 = 'C'; cout prints a char as a character.
68	ch + 1: ch is promoted to int (integral promotion), result is the int 68, so cout prints a number, not a character.
1	bool flag = 5: any non-zero int converts to true (stored as 1); without boolalpha cout prints 1.
true	After std::boolalpha the same bool prints as text; boolalpha is a sticky manipulator.
s >= u	Signed int -1 is converted to unsigned int (value 4294967295) before comparing, so -1 < 1 is false.

Corrected code (Task04/corrected.cpp) using brace initialisation and static_cast; output:

```
3.5 C true
s < u
```

Alternative concept and complexity analysis – all conversions explicit

Solution: write every conversion as static_cast and brace-initialise every variable. Running time and memory do not change: static_cast of the same types produces identical machine code, so both are O(1) per expression with the same space. The benefit is error detection: narrowing in braces (int x{3.7}) is rejected at compile time instead of silently truncating at run time; an explicit cast also documents intent and lets reviewers search for it. The corrected program fixed the signed/unsigned surprise by casting both to long long.

Task 5 – Expressions and Precedence

5.1 Key points

- Precedence high→low (relevant): ++/-- (postfix, prefix) > * / % > + - > < <= > >= > == != > && > || > ?: > = += -=. Associativity: arithmetic is left-to-right; ?: and assignment are right-to-left.
- x++ uses the old value then increments; ++x increments then uses the new value. i = i++ modifies i twice without a sequence point in older standards (undefined behaviour before C++11, unspecified-looking and useless in practice), so it is a mistake.
- Since C++11, / truncates toward zero and the sign of % follows the dividend: -10 / 3 = -3 and -10 % 3 = -1 (check: (-3)*3 + (-1) = -10).
- (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct. && binds tighter than ||, so removing the brackets gives the same result, but the brackets show the two time windows and prevent mistakes when someone edits it.
- Surcharge expression: fare += fare * PEAK_SURCHARGE_PERCENT / 100; multiply first, then divide, so no paise are lost in integer arithmetic.

5.2 Predict the Output

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

My predicted output (write before running):
(write your own prediction here BEFORE running the program)

Actual output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line	Explanation (operators and order)
11 16	x + y * 2: * has higher precedence than +, so 3*2 = 6, then 5 + 6 = 11. (x + y) * 2: brackets first, 8 * 2 = 16.
9 6 4	z = x++ + ++y: x++ yields 5 (then x becomes 6); ++y makes y 4 and yields 4; 5 + 4 = 9 is assigned to z (= lowest precedence, evaluated last). Then x = 6, y = 4.
6 4	(x > y ? x : y): 6 > 4 is true, result x = 6. x % 4 * 2: % and * have equal precedence, left-to-right: (6 % 4) * 2 = 2 * 2 = 4.
3 1 -3 -1	10/3 = 3, 10%3 = 1; -10/3 truncates toward zero = -3; -10%3 takes the sign of the dividend = -1.
18	Loop runs i = 0, 3, 6, 9 (i += 3 compound assignment); k = 0+3+6+9 = 18; i = 12 stops the loop.

Alternative concept and complexity analysis – bool peak[24] lookup table

Solution: bool peak[24] filled once (hours 8,9,17,18,19 true) and checked with peak[hour] after validating 0 <= hour <= 23. Measured (Task05/peak_table.cpp): table and expression give 0 mismatches for all 24 hours; peak hours = 8 9 17 18 19. Time per check is O(1) for both; the expression needs up to 4 comparisons plus logical operators, the table needs one bounds check and one memory read. Extra space for the table: 24 bytes (one-time initialisation cost O(24) = O(1)). The expression is easier to read and has no table to keep in sync; the table is easier to extend if peak windows become irregular.

Task 6 – Control Structures

6.1 Choosing structures

- if–else: for conditions on ranges (peak hours, balance check). switch: for choosing among discrete values such as an enum class zone, where each case is a single value; compiler can warn about unhandled enum values with -Wswitch. ?: : for short one-value choices (peak ? surcharge : 0), not for long logic.
- Looping the journeys: a for loop with an index when the position is needed; a range-based for (for (const Trip& t : trips)) when every element is processed – safest, no off-by-one error, so it is the best choice here; a while loop when the number of journeys is unknown (read until end of input).
- continue skips an invalid journey and goes to the next; break leaves a switch case (otherwise fall-through) or stops the day's processing.
- If no case of a switch matches and there is no default, nothing is executed and control continues after the switch. In my fare() the variable is initialised to 0, so an unknown zone gives fare 0 – so a default case (or validation before the call) is needed in the final design.

6.2 Pseudocode (algorithmic style)

```
ALGORITHM ComputeFare(zone, peak)
  INPUT : zone in {Inner, Middle, Outer}, peak (Boolean)
  OUTPUT: fare in paise (integer)
  BEGIN
    SWITCH zone
      CASE Inner  : base <- BASE_PAISE[0]
      CASE Middle : base <- BASE_PAISE[1]
      CASE Outer  : base <- BASE_PAISE[2]
    END SWITCH
    IF peak THEN
      base <- base + base * PEAK_SURCHARGE_PERCENT / 100
    END IF
    RETURN base
  END

ALGORITHM CanTravel(balance, fare)
  RETURN (balance - fare) >= MIN_BALANCE_PAISE

ALGORITHM ProcessDay(trips)
  FOR EACH trip IN trips DO
    fare <- ComputeFare(trip.zone, trip.peak)
    IF CanTravel(balance, fare) THEN balance <- balance - fare; PRINT fare
    ELSE PRINT "Denied"
  END FOR
```

6.3 Complete the Code – the TODO parts

```
long long fare(Zone z, bool peak)
{
```

```
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;
}
```

Output of the completed program (matches the expected output exactly):

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Check: 10000 − 1000 = 9000 (Rs 90); Outer peak = 3000 + 3000×20/100 = 3600, 9000 − 3600 = 5400 (Rs 54) ≥ 5000 so allowed; Middle = 2000, 5400 − 2000 = 3400 < 5000 so denied.

6.4 Boundary check: exactly 10 AM and 8 PM

```
7:00 -> off-peak
8:00 -> peak
9:00 -> peak
10:00 -> off-peak
16:00 -> off-peak
17:00 -> peak
19:00 -> peak
20:00 -> off-peak
```

10:00 and 20:00 are consistently off-peak because the upper bound is exclusive (< 10, < 20), and 8:00 and 17:00 are peak because the lower bound is inclusive. This fixes the inconsistency in the first gate software.

Alternative concept and complexity analysis – array indexed by zone

Solution: constexpr long long FARE[] = {1000, 2000, 3000}; base = FARE[static_cast<int>(z)]. Time is O(1) for both; a switch over consecutive values is often compiled into a jump table, which is the same as an array read. Space: the array uses 3 x 8 = 24 bytes; the switch stores the constants in the code. For a fourth zone, the array design needs one new element and one enum value; the switch needs a new case line in every switch on the zone. The array risks an out-of-range index for an unknown zone (undefined behaviour), so it needs a bounds check; the switch gives a defined path via default.

Task 7 – From Procedural to Object-Oriented

7.1 Procedural version (Task07/procedural.cpp)

Global balance and free functions fare(), canTravel(). Output:

```
paid 1000 left 9000
paid 3600 left 5400
denied, left 5400
after careless write: -999
```

The last line shows the problem: any code can write balance = -999 and no rule is checked.

7.2 Object-oriented version (Task07/oop.cpp)

```
class SmartCard {
    long long balance_;
    static constexpr long long MIN_BALANCE = 5000;
public:
```

```
    explicit SmartCard(long long b) : balance_{b >= 0 ? b : 0} {}
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return
true; }
    bool recharge(long long a) { if (a <= 0) return false; balance_ += a; return true; }
    long long balance() const { return balance_; }
};
paid 1000 left 9000
paid 3600 left 5400
denied, left 5400
negative recharge accepted? false
```

7.3 Comparison

Criterion	Procedural	Object-oriented
Prevents negative balance	No – balance is global and writable	Yes – balance_ private; deduct() checks canTravel() first
Change minimum balance	Find every function using balance and the constant	Change MIN_BALANCE once inside SmartCard
Multiple cards	Needs arrays of globals	Each object has its own balance_
Testing	Global state leaks between tests	Create a fresh SmartCard per test
Code size	Slightly shorter	Slightly longer (class definition)

Alternative concept and complexity analysis – public balance member

Solution: struct SmartCard { long long balance; }; with deduct() written as a free function. Time and space do not change (O(1), 8 bytes). What breaks: any part of the program can execute card.balance = -999 or card.balance -= 100000 and violate rules R1 and R2; with private balance_ the same line is a compile error (demonstrated in the comment in oop.cpp). The public version also allows two functions to implement different minimum-balance rules for the same card.

Task 8 – Read and Print Data Safely

8.1 Design

- cin >> v returns false in the stream state when letters are typed; if (std::cin >> v) or cin.fail() detects this. The failed text stays in the buffer, so a loop would repeat forever.
- Recovery: std::cin.clear() resets the error flags, then std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n') discards the bad line.
- Money is stored in paise and printed with <iomanip>: f / 100 then '.' then setw(2) with setfill('0') for f % 100, giving Rs 36.00. For doubles, std::fixed << std::setprecision(2).
- The range check (zone 0–2, hour 0–23) is included in the same condition so out-of-range numbers are also rejected.

8.2 Demonstration of recovery (input_invalid.txt: abc, 7, 2, xyz, 9)

```
Zone (0-2):   invalid input, try again
Zone (0-2):   invalid input, try again
Zone (0-2): Hour (0-23):   invalid input, try again
Hour (0-23): ---- Journey report ----
Zone           2
Hour           9
Fare          Rs 36.00
Fare as double: 36.00
```

"abc" fails (letters), "7" fails the range check, "2" is accepted for zone; "xyz" fails, "9" is accepted for hour. Zone 2 (Outer) at 9:00 is peak: 3000 + 600 = Rs 36.00.

Alternative concept and complexity analysis – getline + stoi

Solution (Task08/alt_getline.cpp): read the whole line with std::getline, convert with std::stoi(line, &pos), check pos == line.size() so "12abc" is rejected, and catch std::exception for "abc" (invalid_argument) or huge numbers (out_of_range). It gave the identical result for the same input. Time to process a line of length L is O(L) in both, buffer space O(L) for the string versus O(1) for cin >> int. Recovery: with cin >> you must clear() and ignore(); with getline the stream is never put into a fail state because each line is consumed completely, which is simpler and also catches trailing junk like "12abc" that cin >> int would accept as 12.

Task 9 – Implement and Test the Fare Logic

The test program (Task09/test.cpp) uses a table of cases in a loop. Actual output:

```
Case                      Expected  Actual    Result
Inner off-peak 12:00      1000      1000      PASS
Middle off-peak 12:00     2000      2000      PASS
Outer off-peak 12:00      3000      3000      PASS
Inner peak 8:00           1200      1200      PASS
Middle peak 9:00          2400      2400      PASS
Outer peak 18:00          3600      3600      PASS
Boundary 8:00 (peak)      1200      1200      PASS
Boundary 10:00 (off)      1000      1000      PASS
Boundary 17:00 (peak)     1200      1200      PASS
Boundary 20:00 (off)      1000      1000      PASS
Invalid zone 3            -1        -1        PASS
Invalid hour 25           -1        -1        PASS
Invalid hour -1           -1        -1        PASS
Balance 7000, fare 2000 (leaves 5000): allowed PASS
Balance 7000, fare 2001 (leaves 4999): denied PASS
5-journey total paise: 11000 (hand: 11000) PASS
13/13 table cases passed
```

Hand-calculated 5-journey set

#	Journey	Working	Fare (paise)
1	Inner, 12:00	off-peak	1000
2	Outer, 09:00	3000 + 20% = 3000 + 600	3600
3	Middle, 18:00	2000 + 400	2400
4	Inner, 10:00	10:00 is off-peak	1000
5	Outer, 20:00	20:00 is off-peak	3000
	Total	1000+3600+2400+1000+3000	11000 (Rs 110.00)

Invalid cases: unknown zone (3) and out-of-range hours (25, −1) return −1 (rejected); a negative recharge returns false in Task 7; letters typed for the hour are rejected by the safe input loop in Task 8. All results match the expected values.

Alternative concept and complexity analysis – table-driven tests

Solution: the table of Case{name, zone, hour, expected} looped over in a for loop – this is what Task 9 does. The alternative is separate hand-written checks (if (fare(...) != 1000) ...). Time for t tests is O(t) in both; the table needs O(t) extra space (about 40 bytes per case plus the name string), separate checks need none at run time but add O(t) lines of code. A new boundary case is one new line in the table, versus a new block of code, so the table is easier to extend and less likely to contain copy-paste errors.

Task 10 – Compare Expectations with Execution

Task10/totals.cpp processes n journeys with a repeating mix of 10 fares (including concession fares 1035 and 1815 paise) and accumulates the total in four ways. Measured results:

Journeys	long long (paise)	double total (Rs x 100)	float total error (Rs)	int32 total (paise)
20,000	58,500,000	58,500,000.000000 (error 0)	0	58,500,000 (ok)
500,000	1,462,500,000	1,462,500,000.000000 (error 0)	−10,660	1,462,500,000 (ok, close to limit)
3,000,000	8,775,000,000	8,775,000,000.000000 (error 0)	+30,672	185,065,408 (OVERFLOW)

Findings

- Did double and integer totals agree? For this fare mix, yes – the double total agreed with the integer total to the printed precision (difference below 1e-3 paise). However Task 3 showed that adding 0.10 repeatedly drifts (2e-6 Rs at 3,000,000 additions), so agreement depends on the values; double offers no guarantee.
- Does int overflow? At 3,000,000 journeys the true total is 8,775,000,000 paise but a 32-bit int wrapped to 185,065,408 – wrong by about Rs 85 million. At 500,000 it was still correct (1,462,500,000 < 2,147,483,647). Signed int overflow is undefined behaviour; the wrapped value was obtained with unsigned arithmetic to show the effect safely.
- float failed badly: it has only about 7 significant digits, so the total was off by Rs 30,672 at 3,000,000 journeys.
- Compiler warnings: -Wall -Wextra produced only one warning in the whole assignment – -Wsign-compare in Task04/predict.cpp (comparison of int and unsigned int, meaning the signed value is converted to unsigned and the result may be wrong). All other programs compiled without warnings.

Alternative concept and complexity analysis – int vs long long vs double accumulator

int: fails by overflow once the total exceeds 2,147,483,647 paise (Rs 21.4 million) – here between 500,000 and 3,000,000 journeys. long long: safe up to 9.22 x 10^18 paise, far beyond the network. double: no overflow, but 53-bit mantissa means integers are exact only up to 9.0 x 10^15 and fractions of a rupee are approximate, so precision is lost rather than range. Time per addition is O(1) for all; measured for 10^8 additions: long long 31 ms, double 240 ms (the double loop also converts int to double). Space per variable: 4 bytes (int), 8 bytes (long long), 8 bytes (double).

Task 11 – Recommend and Defend the Final Design

11.1 Recommendation

Decision	Choice	Evidence from my experiments
Money type	long long in paise	double drifted (2e-6 Rs at 3M additions); int overflowed at 3M journeys (185,065,408 vs 8,775,000,000).
Constants	constexpr in one header; enum class Zone	One place to revise; typed; unknown zone is a compile-time error.
Peak rule	(h >= 8 && h < 10) || (h >= 17 && h < 20)	All 24 hours match the table; 8:00/17:00 peak, 10:00/20:00 off-peak.

Decision	Choice	Evidence from my experiments
Classes	SmartCard (private balance), FareTable, Journey	Procedural version accepted balance = -999; the class cannot.
Input	validated cin or getline/stoi	Recovery demonstrated on "abc", "7", "xyz".

11.2 Fare revision procedure

Change the value in the constants header (e.g. BASE_PAISE or PEAK_SURCHARGE_PERCENT), recompile, run the table-driven test (Task 9) and compare against the hand-calculated fares, then deploy. Only one line changes.

11.3 Limitations

- Fares are compiled in, so every revision needs recompilation and redeployment of all gates.
- Concessions are not yet in the code (only designed in Task 1).
- The switch gives fare 0 for an unknown zone unless input is validated first.
- Time is represented as an hour only; minutes (e.g. 9:59 vs 10:00) and public holidays are not handled.
- No persistence or concurrency: 3,000,000 journeys a day (about 35 per second on average, higher at peak) needs thread-safe card updates in real gates.

11.4 Is the design correct for the whole network?

Yes for the arithmetic: per-journey cost is O(1) (a few operations), 3,000,000 journeys are processed in milliseconds, and the long long accumulator holds up to 9.22 x 10^18 paise while the worst day needs 9 x 10^9. The correctness of the fare logic does not depend on scale; the remaining risks (concurrency, network failures) are outside the scope of this assignment.

Alternative concept and complexity analysis – fare table from a configuration file

Solution: at start-up read z fare values from a text file (std::ifstream, parse z integers) into an array. Extra start-up time is O(z) with z = 3 (or 4) – microseconds; space is z x 8 bytes = 24–32 bytes (+ file buffer). Benefit: a fare revision is a file edit and gate restart, with no recompilation or redeployment of the program. Cost: the values are no longer compile-time constants (no constexpr, no static_assert), the file can be missing or contain letters or negative values, so validation and error handling are required. Recommendation: keep constexpr for the rules that change rarely, and move the fare table to a configuration file only if revisions are frequent, with validation at load time.

Task 12 – Day-4 Design Challenge

Changed requirement: fourth zone Airport (Rs 60); a 10% weekend discount that does not combine with the peak surcharge; a monthly pass with unlimited journeys in two chosen zones. Decision: MODIFY (retain the SmartCard and the integer-paise approach; replace switch logic by a table indexed by zone; add a Pass class and a separate discount rule).

A. Impact Analysis

Element	Affected?	Change needed
BASE_PAISE[] constant	Yes	Add 6000 as the fourth element; add ZONE_COUNT = 4.
enum class Zone	Yes	Add Airport.
New constant	Yes	WEEKEND_DISCOUNT_PERCENT = 10.
fare() switch	Yes	Add case Zone::Airport – or replace by table lookup (chosen).

Element	Affected?	Change needed
Peak expression	No	Unchanged, but must not be applied on a weekend.
Time input	Yes	Needs a weekend flag in addition to the hour.
SmartCard::deduct/canTravel	Partly	The Rs 50 rule remains; a pass journey must charge 0.
Pass	New class	Stores two zones; covers(zone).
Test table	Yes	New cases for Airport, weekend and pass.

B. Evaluation of the Original Design

Can the switch absorb the changes? Airport: yes, one more case and constant (works, but it must be added in every switch on Zone – with the array/table only once). Weekend discount: the original fare(Zone, bool peak) has only a peak parameter; a boolean "weekend" cannot be expressed without changing the signature and every call site, and a naive implementation that applies both (surcharge then discount) gives the wrong result. Worked example – Outer, Saturday 9:00: naive both-rules = 3000 + 600 = 3600, then −10% = 3240; the requirement says discount only = 2700. Monthly pass: SmartCard holds only a balance, so passes need a new attached object; adding an isPass flag and zones to SmartCard would mix two concerns. Conclusion: the SmartCard class and the paise/constexpr decisions absorb the change (Retain), the switch-based fare logic needs modification.

C. Adaptation and Implementation

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr int ZONE_COUNT = 4;
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000, 6000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

long long fare(Zone z, const Time& t) {
    long long b = BASE_PAISE[static_cast<int>(z)];
    if (t.weekend) return b - b * WEEKEND_DISCOUNT_PERCENT / 100; // no surcharge
    if (isPeak(t.hour)) return b + b * PEAK_SURCHARGE_PERCENT / 100;
    return b;
}

class Pass { Zone a_, b_;
  public: Pass(Zone a, Zone b) : a_{a}, b_{b} {}
          bool covers(Zone z) const { return z == a_ || z == b_; } };
```

SmartCard::travel(zone, time, charged) first checks an attached pass (charge 0 if the zone is covered), otherwise computes the fare and applies the Rs 50 rule exactly as before. Full code is in Appendix A (Task12/redesign.cpp). The weekend rule is a separate branch so the two rules cannot be combined by mistake. Actual output:

```
Airport weekday off-peak 12h: 6000 (expected 6000) PASS
Airport weekday peak 9h: 7200 (expected 7200) PASS
Outer Saturday 9h (no surcharge): 2700 (expected 2700) PASS
Inner Sunday 18h (discount only): 900 (expected 900) PASS
Middle weekday peak 17h: 2400 (expected 2400) PASS
Pass, Inner peak: allowed=1 charged=0 bal=10000
Pass, Middle peak: allowed=1 charged=0 bal=10000
Pass, Airport off-peak (not covered): allowed=0 charged=0 bal=10000
Pass, Outer (not covered): allowed=1 charged=3000 bal=7000
5/5 fare cases passed
```

The pass covers Inner and Middle, so both journeys are free even in peak hours; the Airport journey (Rs 60) is denied because 10000 − 6000 = 4000 is below the Rs 50 minimum; the Outer journey is charged 3000 (balance 7000 ≥ 5000).

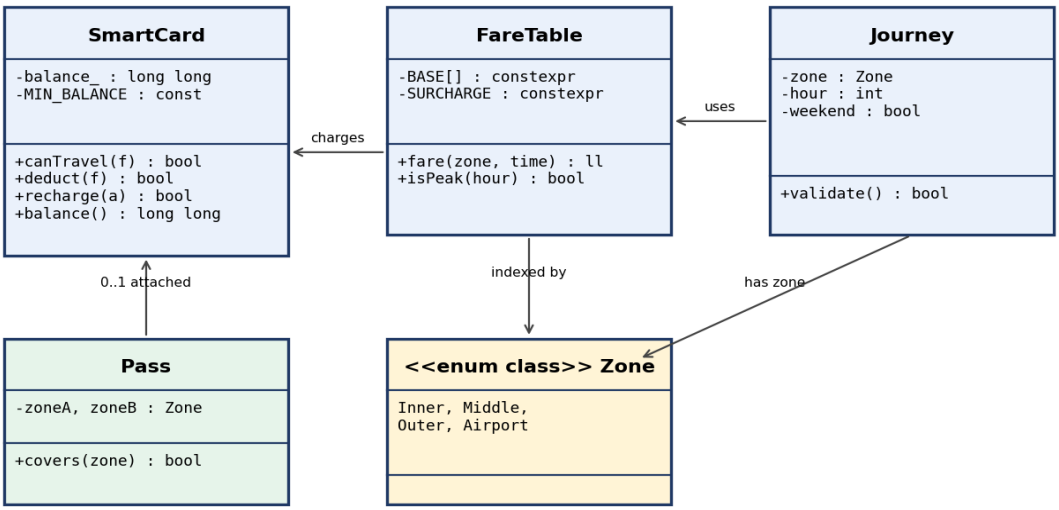
D. Evidence-Based Defence

Option	Pros	Cons	Verdict
Retain unchanged	No work	Cannot express weekend or pass; wrong fare 3240 vs 2700	Rejected
Modify (chosen)	Keeps tested paise logic and Rs 50 rule; fare table indexed by zone; 5/5 new and 13/13 old tests pass	Signature of fare() changes; a Pass object must be attached	Accepted
Full redesign (class hierarchy of Ticket types)	Very flexible	More code, no benefit for these three changes	Not needed

Defence: the evidence from the tests shows that integer paise made the new 10% discount exact (3000 − 300 = 2700, 1000 − 100 = 900); the encapsulated SmartCard still enforces the Rs 50 rule for non-pass journeys; the two-line fare table change and one new class were enough. Trade-off: the design is still compiled-in (see Task 11), and passes currently do not model expiry dates – a monthly pass needs a validity period, which would be the next change.

Figure and repository

Figure 1: UML class diagram (original design + Task 12 additions in green/yellow)



Repository layout and commit plan

```
DSA01-<RegisterNumber>-Assignment1/
  README.md   (name, register no., g++ 13.3.0, command, run steps, result of each task)
  Task03/ types.cpp
  Task04/ predict.cpp  corrected.cpp
  Task05/ predict.cpp  peak_table.cpp
  Task06/ fare.cpp  peak_boundary.cpp
  Task07/ procedural.cpp  oop.cpp
  Task08/ input.cpp  alt_getline.cpp  input_invalid.txt  input_valid.txt
  Task09/ test.cpp
  Task10/ totals.cpp
  Task12/ redesign.cpp
Compile : g++ -std=c++17 -Wall -Wextra file.cpp -o file
Commit one task at a time, e.g. "Task06: complete fare() and canTravel()"
```

Appendix A – Source code

Task03/types.cpp

```
#include <climits>
#include <cstdio>
#include <iomanip>
#include <iostream>
int main()
{
    std::cout << "bool " << sizeof(bool) << ", char " << sizeof(char) << ", int " <<
sizeof(int)
              << ", long long " << sizeof(long long) << ", unsigned " <<
sizeof(unsigned)
              << ", float " << sizeof(float) << ", double " << sizeof(double) << '\n';
    std::cout << "INT_MAX " << INT_MAX << "\nLLONG_MAX " << LLONG_MAX << "\nUINT_MAX "
<< UINT_MAX << '\n';
    // Precision: add Rs 0.10 n times
    for (int n : {10, 1000, 100000, 3000000}) {
        double rs = 0.0; long long paise = 0;
        for (int i = 0; i < n; ++i) { rs += 0.10; paise += 10; }
        std::cout << std::setprecision(15) << "n=" << n << " double=" << rs
                  << " exact=" << paise / 100.0 << " error=" << rs - paise / 100.0 <<
'\n';
    }
    std::cout << std::boolalpha << "0.1+0.2==0.3 ? " << (0.1 + 0.2 == 0.3) << '\n';
    // int overflow for one day's network collection: 3,000,000 journeys * Rs 30 =
9,000,000,000 paise
    long long total = 3000000LL * 3000;
    std::cout << "network max-day paise " << total << " fits int? " << (total <=
INT_MAX) << '\n';
}
```

Task04/corrected.cpp

```
#include <iostream>
int main()
{
    int a{7}, b{2};
    double c{static_cast<double>(a) / b};
    char ch{static_cast<char>('A' + 2)};
    bool flag{5 != 0};
    unsigned int u{1};
    int s{-1};
    // int bad{3.7};  // error: narrowing conversion rejected by braces
    std::cout << c << ' ' << ch << ' ' << std::boolalpha << flag << '\n';
    std::cout << (static_cast<long long>(s) < static_cast<long long>(u) ? "s < u" : "s
>= u") << '\n';
}
```

Task05/peak_table.cpp

```
#include <iostream>
bool peakExpr(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
int main()
{
    bool peak[24] = {};
    for (int h = 0; h < 24; ++h) peak[h] = peakExpr(h);
    int mism = 0;
    for (int h = 0; h < 24; ++h) if (peak[h] != peakExpr(h)) ++mism;
    std::cout << "peak hours:";
    for (int h = 0; h < 24; ++h) if (peak[h]) std::cout << ' ' << h;
    std::cout << "\nmismatches table vs expression: " << mism << '\n';
}
```

Task06/fare.cpp

```
#include <iomanip>
#include <iostream>
```

```
enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000;                        // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << "  balance Rs " << balance / 100 << '\n';
    }
}
```

Task06/peak_boundary.cpp

```
#include <iostream>
bool isPeak(int hour) { return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20); }
int main()
{
    for (int h : {7, 8, 9, 10, 16, 17, 19, 20})
        std::cout << h << ":00 -> " << (isPeak(h) ? "peak" : "off-peak") << '\n';
}
```

Task07/procedural.cpp

```
#include <iostream>
// Procedural style: global state + free functions
long long balance = 10000;               // anyone can write to this
const long long MIN_BALANCE = 5000;
long long fare(int zone, bool peak) {
    long long b = (zone + 1) * 1000;
    return peak ? b + b * 20 / 100 : b;
}
bool canTravel(long long f) { return balance - f >= MIN_BALANCE; }
int main() {
    int zones[] = {0, 2, 1}; bool peaks[] = {false, true, false};
    for (int i = 0; i < 3; ++i) {
        long long f = fare(zones[i], peaks[i]);
        if (canTravel(f)) { balance -= f; std::cout << "paid " << f << " left " <<
balance << '\n'; }
        else std::cout << "denied, left " << balance << '\n';
    }
```

```
    balance = -999;                       // nothing stops this bug
    std::cout << "after careless write: " << balance << '\n';
}
```

Task07/oop.cpp

```
#include <iostream>
enum class Zone { Inner, Middle, Outer };
class FareTable {
    static constexpr long long BASE[3] = {1000, 2000, 3000};
    static constexpr int SURCHARGE = 20;
public:
    long long fare(Zone z, bool peak) const {
        long long b = BASE[static_cast<int>(z)];
        return peak ? b + b * SURCHARGE / 100 : b;
    }
};
class SmartCard {
    long long balance_;
    static constexpr long long MIN_BALANCE = 5000;
public:
    explicit SmartCard(long long b) : balance_{b >= 0 ? b : 0} {}
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return
true; }
    bool recharge(long long a) { if (a <= 0) return false; balance_ += a; return true; }
    long long balance() const { return balance_; }
};
int main() {
    FareTable ft; SmartCard card{10000};
    struct Trip { Zone z; bool p; } trips[] = {{Zone::Inner, false}, {Zone::Outer,
true}, {Zone::Middle, false}};
    for (auto& t : trips) {
        long long f = ft.fare(t.z, t.p);
        if (card.deduct(f)) std::cout << "paid " << f << " left " << card.balance() <<
'\n';
        else std::cout << "denied, left " << card.balance() << '\n';
    }
    std::cout << "negative recharge accepted? " << std::boolalpha << card.recharge(-500)
<< '\n';
    // card.balance_ = -999;   // compile error: balance_ is private
}
```

Task08/input.cpp

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
int readInt(const std::string& prompt, int lo, int hi) {
    int v;
    while (true) {
        std::cout << prompt;
        if (std::cin >> v && v >= lo && v <= hi) return v;
        if (std::cin.eof()) return -1;
        std::cout << "  invalid input, try again\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}
std::string rs(long long p) { std::ostringstream o; o << "Rs " << p / 100 << '.' <<
std::setw(2) << std::setfill('0') << p % 100; return o.str(); }
int main() {
    int zone = readInt("Zone (0-2): ", 0, 2);
    int hour = readInt("Hour (0-23): ", 0, 23);
    long long f = (zone + 1) * 1000LL;
    if ((hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)) f += f * 20 / 100;
    std::cout << "---- Journey report ----\n"
              << std::left << std::setw(10) << "Zone" << std::right << std::setw(6) <<
zone << '\n'
```

```
              << std::left << std::setw(10) << "Hour" << std::right << std::setw(6) <<
hour << '\n'
              << std::left << std::setw(10) << "Fare" << std::right << std::setw(12) <<
rs(f) << '\n';
    std::cout << std::fixed << std::setprecision(2) << "Fare as double: " << f / 100.0
<< '\n';
}
```

Task08/alt_getline.cpp

```
#include <iostream>
#include <string>
int readLine(const std::string& prompt, int lo, int hi) {
    std::string line;
    while (std::cout << prompt, std::getline(std::cin, line)) {
        try {
            size_t pos; int v = std::stoi(line, &pos);
            if (pos == line.size() && v >= lo && v <= hi) return v;
        } catch (const std::exception&) {}
        std::cout << "  invalid input, try again\n";
    }
    return -1;
}
int main() { int z = readLine("Zone (0-2): ", 0, 2); int h = readLine("Hour (0-23): ",
0, 23); std::cout << "zone " << z << " hour " << h << '\n'; }
```

Task09/test.cpp

```
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE[] = {1000, 2000, 3000};
constexpr int SURCHARGE = 20;
constexpr long long MIN_BAL = 5000;
bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
bool validHour(int h) { return h >= 0 && h <= 23; }
long long fare(Zone z, int hour) {
    long long b = BASE[static_cast<int>(z)];
    return isPeak(hour) ? b + b * SURCHARGE / 100 : b;
}
struct Case { std::string name; int zone; int hour; long long expected; };
int main() {
    std::vector<Case> cases = {
        {"Inner off-peak 12:00", 0, 12, 1000}, {"Middle off-peak 12:00", 1, 12, 2000},
{"Outer off-peak 12:00", 2, 12, 3000},
        {"Inner peak 8:00", 0, 8, 1200}, {"Middle peak 9:00", 1, 9, 2400}, {"Outer peak
18:00", 2, 18, 3600},
        {"Boundary 8:00 (peak)", 0, 8, 1200}, {"Boundary 10:00 (off)", 0, 10, 1000},
        {"Boundary 17:00 (peak)", 0, 17, 1200}, {"Boundary 20:00 (off)", 0, 20, 1000},
        {"Invalid zone 3", 3, 12, -1}, {"Invalid hour 25", 0, 25, -1}, {"Invalid hour -
1", 0, -1, -1}};
    int pass = 0;
    std::cout << std::left << std::setw(26) << "Case" << std::setw(10) << "Expected" <<
std::setw(10) << "Actual" << "Result\n";
    for (const Case& c : cases) {
        long long actual = (c.zone < 0 || c.zone > 2 || !validHour(c.hour)) ? -1 :
fare(static_cast<Zone>(c.zone), c.hour);
        bool ok = actual == c.expected; pass += ok;
        std::cout << std::setw(26) << c.name << std::setw(10) << c.expected <<
std::setw(10) << actual << (ok ? "PASS" : "FAIL") << '\n';
    }
    // Balance boundary
    long long bal = 7000;  bool t1 = bal - 2000 >= MIN_BAL;   // leaves exactly Rs 50 ->
allowed
    bool t2 = bal - 2001 >= MIN_BAL;                          // leaves Rs 49.99 ->
denied
    std::cout << "Balance 7000, fare 2000 (leaves 5000): " << (t1 ? "allowed PASS" :
"denied FAIL") << '\n';
    std::cout << "Balance 7000, fare 2001 (leaves 4999): " << (!t2 ? "denied PASS" :
"allowed FAIL") << '\n';
```

```
    // 5-journey hand total: Inner 12:00 (1000) + Outer 9:00 (3600) + Middle 18:00
(2400) + Inner 10:00 (1000) + Outer 20:00 (3000) = 11000
    long long total = fare(Zone::Inner, 12) + fare(Zone::Outer, 9) + fare(Zone::Middle,
18) + fare(Zone::Inner, 10) + fare(Zone::Outer, 20);
    std::cout << "5-journey total paise: " << total << " (hand: 11000) " << (total ==
11000 ? "PASS" : "FAIL") << '\n';
    std::cout << pass << "/" << cases.size() << " table cases passed\n";
}
```

Task10/totals.cpp

```
#include <chrono>
#include <climits>
#include <cstdint>
#include <iomanip>
#include <iostream>
int main() {
    const long long fares[] = {1000, 1200, 2000, 2400, 3000, 3600, 6000, 7200, 1035,
1815}; // paise, incl. concession fares
    for (long long n : {20000LL, 500000LL, 3000000LL}) {
        long long exact = 0; double dbl = 0.0; float flt = 0.0f; uint32_t wrap = 0;
        for (long long i = 0; i < n; ++i) {
            long long f = fares[i % 10];
            exact += f; dbl += f / 100.0; flt += f / 100.0f; wrap +=
static_cast<uint32_t>(f);
        }
        std::cout << "n=" << n << "\n  long long paise = " << exact
                  << "\n  double Rs*100       = " << std::fixed << std::setprecision(6)
<< dbl * 100
                  << "  diff(Rs)=" << std::scientific << std::setprecision(3) << dbl -
exact / 100.0 << std::fixed << std::setprecision(6)
                  << "\n  float Rs            = " << flt << "  diff(Rs)=" << flt - exact
/ 100.0
                  << "\n  int32 wrapped paise = " << static_cast<int32_t>(wrap)
                  << (exact > INT_MAX ? "  <-- OVERFLOW" : "  ok") << '\n';
    }
    auto t0 = std::chrono::steady_clock::now(); long long s = 0; for (long long i = 0; i
< 100000000LL; ++i) s += i & 7;
    auto t1 = std::chrono::steady_clock::now(); double d = 0; for (long long i = 0; i <
100000000LL; ++i) d += static_cast<double>(i & 7);
    auto t2 = std::chrono::steady_clock::now();
    using ms = std::chrono::milliseconds;
    std::cout << "1e8 additions: long long " << std::chrono::duration_cast<ms>(t1 -
t0).count() << " ms, double "
              << std::chrono::duration_cast<ms>(t2 - t1).count() << " ms (checksums " <<
s << ' ' << d << ")\n";
}
```

Task12/redesign.cpp

```
#include <iostream>
#include <string>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr int ZONE_COUNT = 4;
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000, 6000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;

struct Time { int hour; bool weekend; };
bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }

// Rule: weekend discount and peak surcharge never combine; weekend wins on a weekend.
long long fare(Zone z, const Time& t) {
    long long b = BASE_PAISE[static_cast<int>(z)];
    if (t.weekend) return b - b * WEEKEND_DISCOUNT_PERCENT / 100;
    if (isPeak(t.hour)) return b + b * PEAK_SURCHARGE_PERCENT / 100;
    return b;
}

class Pass {                        // unlimited journeys in two chosen zones
```

```
    Zone a_, b_;
public:
    Pass(Zone a, Zone b) : a_{a}, b_{b} {}
    bool covers(Zone z) const { return z == a_ || z == b_; }
};

class SmartCard {
    long long balance_;
    const Pass* pass_ = nullptr;
public:
    explicit SmartCard(long long b) : balance_{b} {}
    void attach(const Pass* p) { pass_ = p; }
    bool travel(Zone z, const Time& t, long long& charged) {
        if (pass_ && pass_->covers(z)) { charged = 0; return true; }
        long long f = fare(z, t);
        if (balance_ - f < MIN_BALANCE_PAISE) { charged = 0; return false; }
        balance_ -= f; charged = f; return true;
    }
    long long balance() const { return balance_; }
};

int main() {
    struct Row { const char* name; Zone z; Time t; long long expected; };
    Row rows[] = {
        {"Airport weekday off-peak 12h", Zone::Airport, {12, false}, 6000},
        {"Airport weekday peak 9h",      Zone::Airport, {9,  false}, 7200},
        {"Outer Saturday 9h (no surcharge)", Zone::Outer, {9, true}, 2700},
        {"Inner Sunday 18h (discount only)", Zone::Inner, {18, true}, 900},
        {"Middle weekday peak 17h",      Zone::Middle,  {17, false}, 2400},
    };
    int ok = 0;
    for (auto& r : rows) { long long f = fare(r.z, r.t); bool p = f == r.expected; ok +=
p;
        std::cout << r.name << ": " << f << " (expected " << r.expected << ") " << (p ?
"PASS" : "FAIL") << '\n'; }
    Pass pass{Zone::Inner, Zone::Middle};
    SmartCard card{10000}; card.attach(&pass);
    long long c;
    bool a = card.travel(Zone::Inner, {9, false}, c);   std::cout << "Pass, Inner peak:
allowed=" << a << " charged=" << c << " bal=" << card.balance() << '\n';
    bool b = card.travel(Zone::Middle, {18, false}, c); std::cout << "Pass, Middle peak:
allowed=" << b << " charged=" << c << " bal=" << card.balance() << '\n';
    bool d = card.travel(Zone::Airport, {12, false}, c);std::cout << "Pass, Airport off-
peak (not covered): allowed=" << d << " charged=" << c << " bal=" << card.balance() <<
'\n';
    bool e = card.travel(Zone::Outer, {12, false}, c);  std::cout << "Pass, Outer (not
covered, low bal): allowed=" << e << " charged=" << c << " bal=" << card.balance() <<
'\n';
    std::cout << ok << "/5 fare cases passed\n";
}
```